

Top 100 DSA Interview Questions for Freshers

Data Structures and Algorithms (DSA) are the backbone of technical interviews. This document provides the top 100 DSA interview questions for freshers in 2025, with complete Python implementations for each, organized by topic—arrays, strings, linked lists, stacks, queues, trees, graphs, sorting, searching, and dynamic programming. Each question includes a brief explanation and a working code solution.

1 Arrays

Arrays store elements in contiguous memory locations, tested for their simplicity and versatility.

1.0.1 1. What is an array?

A collection of elements of the same type stored at contiguous memory locations, accessed via indices.

```
1 # Example array
2 arr = [1, 2, 3, 4, 5]
```

1.0.2 2. How do you find the largest element in an unsorted array?

Iterate through the array, tracking the maximum value. Time complexity: $O(n)$.

```
1 def find_largest(arr):
2     if not arr:
3         return None
4     max_val = arr[0]
5     for num in arr:
6         if num > max_val:
7             max_val = num
8     return max_val
```

1.0.3 3. How do you find the smallest element in an unsorted array?

Track the minimum value while iterating. Time complexity: $O(n)$.

```
1 def find_smallest(arr):
2     if not arr:
3         return None
4     min_val = arr[0]
5     for num in arr:
6         if num < min_val:
7             min_val = num
8     return min_val
```

1.0.4 4. How do you find the second largest element in an array?

Track the largest and second largest values. Time complexity: $O(n)$.

```
1 def second_largest(arr):
2     if len(arr) < 2:
3         return None
4     largest = second = float('-inf')
5     for num in arr:
6         if num > largest:
7             second = largest
8             largest = num
9     elif num > second and num != largest:
```

```

10         second = num
11     return second if second != float('-inf') else None

```

1.0.5 5. How do you find a missing number in an array of 1 to n?

Calculate the expected sum and subtract the actual sum. Time complexity: $O(n)$.

```

1 def find_missing_number(arr, n):
2     expected_sum = n * (n + 1) // 2
3     actual_sum = sum(arr)
4     return expected_sum - actual_sum

```

1.0.6 6. How do you find duplicate numbers in an array?

Use a hash set to track seen numbers. Time complexity: $O(n)$.

```

1 def find_duplicates(arr):
2     seen = set()
3     duplicates = set()
4     for num in arr:
5         if num in seen:
6             duplicates.add(num)
7         else:
8             seen.add(num)
9     return list(duplicates)

```

1.0.7 7. How do you remove duplicates from an array in place?

Use two pointers to keep unique elements. Time complexity: $O(n)$.

```

1 def remove_duplicates(arr):
2     if not arr:
3         return 0
4     write_index = 1
5     for i in range(1, len(arr)):
6         if arr[i] != arr[write_index - 1]:
7             arr[write_index] = arr[i]
8             write_index += 1
9     return write_index

```

1.0.8 8. How do you reverse an array in place?

Swap elements from start and end. Time complexity: $O(n/2)$.

```

1 def reverse_array(arr):
2     left, right = 0, len(arr) - 1
3     while left < right:
4         arr[left], arr[right] = arr[right], arr[left]
5         left += 1
6         right -= 1
7     return arr

```

1.0.9 9. How do you find all pairs in an array that sum to a given number?

Use a hash set to store complements. Time complexity: $O(n)$.

```

1 def find_pairs_with_sum(arr, target):
2     seen = set()
3     pairs = []
4     for num in arr:
5         complement = target - num

```

```

6         if complement in seen:
7             pairs.append((complement, num))
8             seen.add(num)
9     return pairs

```

1.0.10 10. How do you rotate an array by k positions?

Reverse the entire array, then reverse first k and rest. Time complexity: $O(n)$.

```

1 def rotate_array(arr, k):
2     def reverse(arr, start, end):
3         while start < end:
4             arr[start], arr[end] = arr[end], arr[start]
5             start += 1
6             end -= 1
7     k = k % len(arr)
8     reverse(arr, 0, len(arr) - 1)
9     reverse(arr, 0, k - 1)
10    reverse(arr, k, len(arr) - 1)
11    return arr

```

1.0.11 11. How do you find the maximum subarray sum (Kadane's Algorithm)?

Track current and maximum sums, resetting if negative. Time complexity: $O(n)$.

```

1 def max_subarray_sum(arr):
2     max_sum = curr_sum = arr[0]
3     for num in arr[1:]:
4         curr_sum = max(num, curr_sum + num)
5         max_sum = max(max_sum, curr_sum)
6     return max_sum

```

1.0.12 12. How do you find the intersection of two arrays?

Use a hash set for one array and check matches. Time complexity: $O(n)$.

```

1 def array_intersection(arr1, arr2):
2     seen = set(arr1)
3     intersection = []
4     for num in arr2:
5         if num in seen:
6             intersection.append(num)
7             seen.remove(num)
8     return intersection

```

1.0.13 13. How do you move all zeros to the end of an array?

Use two pointers to place non-zeros at front. Time complexity: $O(n)$.

```

1 def move_zeros(arr):
2     non_zero_pos = 0
3     for i in range(len(arr)):
4         if arr[i] != 0:
5             arr[non_zero_pos], arr[i] = arr[i], arr[non_zero_pos]
6             non_zero_pos += 1
7     return arr

```

1.0.14 14. How do you check if an array is sorted?

Compare adjacent elements. Time complexity: $O(n)$.

```

1 def is_sorted(arr):
2     for i in range(len(arr) - 1):
3         if arr[i] > arr[i + 1]:
4             return False
5     return True

```

1.0.15 15. How do you find the kth largest element in an array?

Use a min-heap of size k. Time complexity: $O(n \log k)$.

```

1 import heapq
2
3 def kth_largest(arr, k):
4     heap = []
5     for num in arr:
6         heapq.heappush(heap, num)
7         if len(heap) > k:
8             heapq.heappop(heap)
9     return heap[0]

```

2 Strings

Strings are arrays of characters, tested for manipulation and pattern-matching.

2.0.1 16. How do you reverse a string?

Swap characters from ends. Time complexity: $O(n)$.

```

1 def reverse_string(s):
2     s = list(s)
3     left, right = 0, len(s) - 1
4     while left < right:
5         s[left], s[right] = s[right], s[left]
6         left += 1
7         right -= 1
8     return ''.join(s)

```

2.0.2 17. How do you check if two strings are anagrams?

Use a frequency array or hash map. Time complexity: $O(n)$.

```

1 def are_anagrams(s1, s2):
2     if len(s1) != len(s2):
3         return False
4     freq = [0] * 26
5     for c1, c2 in zip(s1, s2):
6         freq[ord(c1.lower()) - ord('a')] += 1
7         freq[ord(c2.lower()) - ord('a')] -= 1
8     return all(count == 0 for count in freq)

```

2.0.3 18. How do you find the first non-repeating character in a string?

Use a hash map for frequency and first index. Time complexity: $O(n)$.

```

1 def first_non_repeating(s):
2     freq = {}
3     for i, c in enumerate(s):
4         freq[c] = freq.get(c, [0, i])[0] + 1, freq.get(c, [0, i])[1]
5     for c, (count, _) in freq.items():
6         if count == 1:

```

```
7         return c
8     return None
```

2.0.4 19. How do you check if a string is a palindrome?

Compare characters from both ends. Time complexity: $O(n)$.

```
1 def is_palindrome(s):
2     s = ''.join(c.lower() for c in s if c.isalnum())
3     left, right = 0, len(s) - 1
4     while left < right:
5         if s[left] != s[right]:
6             return False
7         left += 1
8         right -= 1
9     return True
```

2.0.5 20. How do you find the length of the longest substring without repeating characters?

Use a sliding window with a hash set. Time complexity: $O(n)$.

```
1 def longest_substring_without_repeating(s):
2     seen = {}
3     max_len = start = 0
4     for i, c in enumerate(s):
5         if c in seen and seen[c] >= start:
6             start = seen[c] + 1
7         else:
8             max_len = max(max_len, i - start + 1)
9             seen[c] = i
10    return max_len
```

2.0.6 21. How do you remove duplicate characters from a string?

Use a hash set to track seen characters. Time complexity: $O(n)$.

```
1 def remove_duplicate_chars(s):
2     seen = set()
3     result = []
4     for c in s:
5         if c not in seen:
6             seen.add(c)
7             result.append(c)
8     return ''.join(result)
```

2.0.7 22. How do you convert a string to an integer (atoi)?

Parse signs, digits, and handle overflow. Time complexity: $O(n)$.

```
1 def atoi(s):
2     s = s.strip()
3     if not s:
4         return 0
5     sign = 1
6     if s[0] in ['+', '-']:
7         sign = -1 if s[0] == '-' else 1
8         s = s[1:]
9     result = 0
10    for c in s:
11        if not c.isdigit():
12            break
```

```

13     result = result * 10 + int(c)
14     result *= sign
15     return max(min(result, 2**31 - 1), -2**31)

```

2.0.8 23. How do you find all permutations of a string?

Use backtracking to generate all arrangements. Time complexity: $O(n!)$.

```

1 def permutations(s):
2     def backtrack(s, path, result):
3         if not s:
4             result.append(''.join(path))
5             return
6         for i in range(len(s)):
7             backtrack(s[:i] + s[i+1:], path + [s[i]], result)
8
9     result = []
10    backtrack(s, [], result)
11    return result

```

2.0.9 24. How do you check if a string is a valid parenthesis sequence?

Use a stack to match parentheses. Time complexity: $O(n)$.

```

1 def is_valid_parentheses(s):
2     stack = []
3     for c in s:
4         if c in '([':
5             stack.append(c)
6         elif c in ')]':
7             if not stack:
8                 return False
9             top = stack.pop()
10            if (c == ')' and top != '(') or (c == ']' and top != '[') or (c == '}' and top != '{'):
11                return False
12    return len(stack) == 0

```

2.0.10 25. How do you find the longest palindromic substring?

Expand around the center for each character. Time complexity: $O(n^2)$.

```

1 def longest_palindromic_substring(s):
2     def expand_around_center(left, right):
3         while left >= 0 and right < len(s) and s[left] == s[right]:
4             left -= 1
5             right += 1
6         return s[left + 1:right]
7
8     longest = ""
9     for i in range(len(s)):
10        pal1 = expand_around_center(i, i)
11        pal2 = expand_around_center(i, i + 1)
12        if len(pal1) > len(longest):
13            longest = pal1
14        if len(pal2) > len(longest):
15            longest = pal2
16    return longest

```

3 Linked Lists

Linked lists are linear data structures with nodes connected by pointers.

3.0.1 26. What is a linked list?

A sequence of nodes with data and a pointer to the next node.

```
1 class ListNode:
2     def __init__(self, val=0, next=None):
3         self.val = val
4         self.next = next
```

3.0.2 27. How do you reverse a linked list?

Update next pointers to reverse direction. Time complexity: $O(n)$.

```
1 class ListNode:
2     def __init__(self, val=0, next=None):
3         self.val = val
4         self.next = next
5
6 def reverse_list(head):
7     prev, curr = None, head
8     while curr:
9         next_temp = curr.next
10        curr.next = prev
11        prev = curr
12        curr = next_temp
13    return prev
```

3.0.3 28. How do you detect a cycle in a linked list?

Use Floyd's Tortoise and Hare algorithm. Time complexity: $O(n)$.

```
1 def has_cycle(head):
2     if not head:
3         return False
4     slow = fast = head
5     while fast and fast.next:
6         slow = slow.next
7         fast = fast.next.next
8         if slow == fast:
9             return True
10    return False
```

3.0.4 29. How do you find the middle node of a linked list?

Use two pointers, fast moves twice as fast. Time complexity: $O(n)$.

```
1 def middle_node(head):
2     slow = fast = head
3     while fast and fast.next:
4         slow = slow.next
5         fast = fast.next.next
6     return slow
```

3.0.5 30. How do you merge two sorted linked lists?

Compare nodes and build a new list. Time complexity: $O(n)$.

```

1 def merge_two_lists(l1, l2):
2     dummy = ListNode(0)
3     curr = dummy
4     while l1 and l2:
5         if l1.val <= l2.val:
6             curr.next = l1
7             l1 = l1.next
8         else:
9             curr.next = l2
10            l2 = l2.next
11            curr = curr.next
12    curr.next = l1 or l2
13    return dummy.next

```

3.0.6 31. How do you find the intersection point of two linked lists?

Equalize lengths and find the common node. Time complexity: $O(n)$.

```

1 def get_intersection_node(headA, headB):
2     if not headA or not headB:
3         return None
4     a, b = headA, headB
5     while a != b:
6         a = a.next if a else headB
7         b = b.next if b else headA
8     return a

```

3.0.7 32. How do you remove the nth node from the end of a linked list?

Use two pointers with a gap of n. Time complexity: $O(n)$.

```

1 def remove_nth_from_end(head, n):
2     dummy = ListNode(0, head)
3     fast = slow = dummy
4     for _ in range(n):
5         fast = fast.next
6     while fast.next:
7         slow = slow.next
8         fast = fast.next
9     slow.next = slow.next.next
10    return dummy.next

```

3.0.8 33. How do you check if a linked list is a palindrome?

Find middle, reverse second half, and compare. Time complexity: $O(n)$.

```

1 def is_palindrome(head):
2     def reverse(head):
3         prev, curr = None, head
4         while curr:
5             next_temp = curr.next
6             curr.next = prev
7             prev = curr
8             curr = next_temp
9         return prev
10
11    slow = fast = head
12    while fast and fast.next:
13        slow = slow.next
14        fast = fast.next.next

```

```

15 second_half = reverse(slow)
16 while second_half and head:
17     if second_half.val != head.val:
18         return False
19     second_half = second_half.next
20     head = head.next
21 return True

```

3.0.9 34. How do you delete a node with only a pointer to that node?

Copy next node's data and skip it. Time complexity: $O(1)$.

```

1 def delete_node(node):
2     node.val = node.next.val
3     node.next = node.next.next

```

3.0.10 35. How do you implement a doubly linked list?

Each node has next and previous pointers.

```

1 class DoublyListNode:
2     def __init__(self, val=0, prev=None, next=None):
3         self.val = val
4         self.prev = prev
5         self.next = next

```

4 Stacks

Stacks follow Last In, First Out (LIFO) principle.

4.0.1 36. What is a stack?

A data structure where elements are added/removed from the top (LIFO).

```

1 class Stack:
2     def __init__(self):
3         self.items = []

```

4.0.2 37. How do you implement a stack using an array?

Use an array with push/pop at the end. Time complexity: $O(1)$.

```

1 class Stack:
2     def __init__(self):
3         self.items = []
4
5     def push(self, item):
6         self.items.append(item)
7
8     def pop(self):
9         return self.items.pop() if self.items else None
10
11     def peek(self):
12         return self.items[-1] if self.items else None

```

4.0.3 38. How do you implement a stack using a linked list?

Push/pop at the head. Time complexity: $O(1)$.

```

1 class StackNode:
2     def __init__(self, val=0, next=None):
3         self.val = val
4         self.next = next
5
6 class Stack:
7     def __init__(self):
8         self.top = None
9
10    def push(self, val):
11        self.top = StackNode(val, self.top)
12
13    def pop(self):
14        if not self.top:
15            return None
16        val = self.top.val
17        self.top = self.top.next
18        return val

```

4.0.4 39. How do you check for balanced parentheses using a stack?

Push opening brackets, pop for matching closing. Time complexity: $O(n)$.

```

1 def is_balanced_parentheses(s):
2     stack = []
3     for c in s:
4         if c in '([{' :
5             stack.append(c)
6         elif c in ')]}' :
7             if not stack:
8                 return False
9             top = stack.pop()
10            if (c == ')' and top != '(') or (c == ']' and top != '[') or (c
11               == '}' and top != '{'):
12                return False
13    return len(stack) == 0

```

4.0.5 40. How do you evaluate a postfix expression?

Use a stack for operands and operators. Time complexity: $O(n)$.

```

1 def evaluate_postfix(expr):
2     stack = []
3     for token in expr.split():
4         if token in '+-*/':
5             b = stack.pop()
6             a = stack.pop()
7             if token == '+':
8                 stack.append(a + b)
9             elif token == '-':
10                stack.append(a - b)
11            elif token == '*':
12                stack.append(a * b)
13            elif token == '/':
14                stack.append(a / b)
15        else:
16            stack.append(int(token))
17    return stack[0]

```

4.0.6 41. How do you find the next greater element in an array?

Use a stack to track elements. Time complexity: $O(n)$.

```
1 def next_greater_element(arr):
2     stack = []
3     result = [-1] * len(arr)
4     for i in range(len(arr)):
5         while stack and arr[stack[-1]] < arr[i]:
6             result[stack.pop()] = arr[i]
7         stack.append(i)
8     return result
```

4.0.7 42. How do you implement two stacks in one array?

Use two pointers from opposite ends. Time complexity: $O(1)$.

```
1 class TwoStacks:
2     def __init__(self, size):
3         self.size = size
4         self.arr = [None] * size
5         self.top1 = -1
6         self.top2 = size
7
8     def push1(self, val):
9         if self.top1 < self.top2 - 1:
10             self.top1 += 1
11             self.arr[self.top1] = val
12
13     def push2(self, val):
14         if self.top1 < self.top2 - 1:
15             self.top2 -= 1
16             self.arr[self.top2] = val
17
18     def pop1(self):
19         if self.top1 >= 0:
20             val = self.arr[self.top1]
21             self.top1 -= 1
22             return val
23         return None
24
25     def pop2(self):
26         if self.top2 < self.size:
27             val = self.arr[self.top2]
28             self.top2 += 1
29             return val
30         return None
```

4.0.8 43. How do you reverse a stack without extra space?

Use recursion to insert at bottom. Time complexity: $O(n^2)$.

```
1 class Stack:
2     def __init__(self):
3         self.items = []
4
5     def push(self, item):
6         self.items.append(item)
7
8     def pop(self):
9         return self.items.pop() if self.items else None
10
```

```

11     def reverse(self):
12         if self.items:
13             temp = self.pop()
14             self.reverse()
15             self._insert_at_bottom(temp)
16
17     def _insert_at_bottom(self, item):
18         if not self.items:
19             self.push(item)
20         else:
21             temp = self.pop()
22             self._insert_at_bottom(item)
23             self.push(temp)

```

4.0.9 44. How do you find the minimum element in a stack in O(1) time?

Maintain a second stack for minimums. Time complexity: O(1).

```

1 class MinStack:
2     def __init__(self):
3         self.stack = []
4         self.min_stack = []
5
6     def push(self, val):
7         self.stack.append(val)
8         if not self.min_stack or val <= self.min_stack[-1]:
9             self.min_stack.append(val)
10
11     def pop(self):
12         if self.stack:
13             val = self.stack.pop()
14             if val == self.min_stack[-1]:
15                 self.min_stack.pop()
16             return val
17         return None
18
19     def get_min(self):
20         return self.min_stack[-1] if self.min_stack else None

```

4.0.10 45. How do you implement a stack using two queues?

Rotate elements to simulate LIFO. Time complexity: O(n) for push.

```

1 from collections import deque
2
3 class Stack:
4     def __init__(self):
5         self.q1 = deque()
6         self.q2 = deque()
7
8     def push(self, val):
9         self.q2.append(val)
10        while self.q1:
11            self.q2.append(self.q1.popleft())
12        self.q1, self.q2 = self.q2, self.q1
13
14    def pop(self):
15        return self.q1.popleft() if self.q1 else None

```

5 Queues

Queues follow First In, First Out (FIFO) principle.

5.0.1 46. What is a queue?

A data structure where elements are added at rear and removed from front.

```
1 from collections import deque
2
3 queue = deque()
```

5.0.2 47. How do you implement a queue using an array?

Use front and rear pointers. Time complexity: $O(1)$.

```
1 class Queue:
2     def __init__(self, size):
3         self.size = size
4         self.arr = [None] * size
5         self.front = self.rear = -1
6
7     def enqueue(self, val):
8         if self.rear < self.size - 1:
9             if self.front == -1:
10                 self.front = 0
11                 self.rear += 1
12                 self.arr[self.rear] = val
13
14     def dequeue(self):
15         if self.front != -1:
16             val = self.arr[self.front]
17             self.front += 1
18             if self.front > self.rear:
19                 self.front = self.rear = -1
20             return val
21         return None
```

5.0.3 48. How do you implement a queue using a linked list?

Use front and rear pointers. Time complexity: $O(1)$.

```
1 class QueueNode:
2     def __init__(self, val=0, next=None):
3         self.val = val
4         self.next = next
5
6 class Queue:
7     def __init__(self):
8         self.front = self.rear = None
9
10    def enqueue(self, val):
11        node = QueueNode(val)
12        if not self.rear:
13            self.front = self.rear = node
14        else:
15            self.rear.next = node
16            self.rear = node
17
18    def dequeue(self):
19        if not self.front:
20            return None
```

```

21     val = self.front.val
22     self.front = self.front.next
23     if not self.front:
24         self.rear = None
25     return val

```

5.0.4 49. How do you implement a circular queue?

Use modulo for wrap-around. Time complexity: $O(1)$.

```

1 class CircularQueue:
2     def __init__(self, size):
3         self.size = size
4         self.arr = [None] * size
5         self.front = self.rear = -1
6
7     def enqueue(self, val):
8         if (self.rear + 1) % self.size == self.front:
9             return False
10        if self.front == -1:
11            self.front = 0
12        self.rear = (self.rear + 1) % self.size
13        self.arr[self.rear] = val
14        return True
15
16    def dequeue(self):
17        if self.front == -1:
18            return None
19        val = self.arr[self.front]
20        if self.front == self.rear:
21            self.front = self.rear = -1
22        else:
23            self.front = (self.front + 1) % self.size
24        return val

```

5.0.5 50. How do you implement a priority queue?

Use a heap for prioritization. Time complexity: $O(\log n)$.

```

1 import heapq
2
3 class PriorityQueue:
4     def __init__(self):
5         self.heap = []
6
7     def push(self, val):
8         heapq.heappush(self.heap, val)
9
10    def pop(self):
11        return heapq.heappop(self.heap) if self.heap else None

```

5.0.6 51. How do you implement a queue using two stacks?

Use one stack for enqueue, another for dequeue. Time complexity: $O(n)$ for dequeue.

```

1 class Queue:
2     def __init__(self):
3         self.s1 = []
4         self.s2 = []
5
6     def enqueue(self, val):

```

```

7         self.s1.append(val)
8
9     def dequeue(self):
10         if not self.s2:
11             while self.s1:
12                 self.s2.append(self.s1.pop())
13         return self.s2.pop() if self.s2 else None

```

5.0.7 52. How do you reverse a queue?

Use a stack to reverse order. Time complexity: $O(n)$.

```

1 from collections import deque
2
3 def reverse_queue(queue):
4     stack = []
5     while queue:
6         stack.append(queue.popleft())
7     while stack:
8         queue.append(stack.pop())
9     return queue

```

5.0.8 53. What is a dequeue (double-ended queue)?

A queue allowing insertion/deletion at both ends.

```

1 from collections import deque
2
3 deq = deque()

```

5.0.9 54. How do you implement a sliding window maximum using a deque?

Store indices of potential maximums. Time complexity: $O(n)$.

```

1 from collections import deque
2
3 def max_sliding_window(arr, k):
4     deq = deque()
5     result = []
6     for i in range(len(arr)):
7         while deq and deq[0] <= i - k:
8             deq.popleft()
9         while deq and arr[deq[-1]] < arr[i]:
10             deq.pop()
11         deq.append(i)
12         if i >= k - 1:
13             result.append(arr[deq[0]])
14     return result

```

5.0.10 55. How do you generate binary numbers from 1 to n using a queue?

Enqueue "1", append 0 and 1. Time complexity: $O(n)$.

```

1 from collections import deque
2
3 def generate_binary_numbers(n):
4     queue = deque(["1"])
5     result = []
6     while len(result) < n:
7         num = queue.popleft()
8         result.append(num)

```

```

9         queue.append(num + "0")
10        queue.append(num + "1")
11    return result[:n]

```

6 Trees

Trees are hierarchical data structures for traversal and manipulation.

6.0.1 56. What is a binary tree?

A tree where each node has at most two children.

```

1 class TreeNode:
2     def __init__(self, val=0, left=None, right=None):
3         self.val = val
4         self.left = left
5         self.right = right

```

6.0.2 57. How do you perform an inorder traversal of a binary tree?

Visit left, root, right. Time complexity: $O(n)$.

```

1 def inorder_traversal(root):
2     result = []
3     def dfs(node):
4         if not node:
5             return
6         dfs(node.left)
7         result.append(node.val)
8         dfs(node.right)
9     dfs(root)
10    return result

```

6.0.3 58. How do you perform a preorder traversal of a binary tree?

Visit root, left, right. Time complexity: $O(n)$.

```

1 def preorder_traversal(root):
2     result = []
3     def dfs(node):
4         if not node:
5             return
6         result.append(node.val)
7         dfs(node.left)
8         dfs(node.right)
9     dfs(root)
10    return result

```

6.0.4 59. How do you perform a postorder traversal of a binary tree?

Visit left, right, root. Time complexity: $O(n)$.

```

1 def postorder_traversal(root):
2     result = []
3     def dfs(node):
4         if not node:
5             return
6         dfs(node.left)
7         dfs(node.right)
8         result.append(node.val)

```

```

9     dfs(root)
10    return result

```

6.0.5 60. How do you find the height of a binary tree?

Compute max height of subtrees. Time complexity: $O(n)$.

```

1 def height(root):
2     if not root:
3         return 0
4     return 1 + max(height(root.left), height(root.right))

```

6.0.6 61. How do you check if a binary tree is balanced?

Check height difference of subtrees. Time complexity: $O(n)$.

```

1 def is_balanced(root):
2     def check_height(node):
3         if not node:
4             return 0
5         left = check_height(node.left)
6         if left == -1:
7             return -1
8         right = check_height(node.right)
9         if right == -1 or abs(left - right) > 1:
10            return -1
11        return max(left, right) + 1
12    return check_height(root) != -1

```

6.0.7 62. How do you find the lowest common ancestor in a binary tree?

Traverse until nodes are in different subtrees. Time complexity: $O(n)$.

```

1 def lowest_common_ancestor(root, p, q):
2     if not root or root == p or root == q:
3         return root
4     left = lowest_common_ancestor(root.left, p, q)
5     right = lowest_common_ancestor(root.right, p, q)
6     if left and right:
7         return root
8     return left or right

```

6.0.8 63. How do you check if a binary tree is a binary search tree (BST)?

Ensure values are in valid range. Time complexity: $O(n)$.

```

1 def is_valid_bst(root):
2     def validate(node, min_val, max_val):
3         if not node:
4             return True
5         if node.val <= min_val or node.val >= max_val:
6             return False
7         return validate(node.left, min_val, node.val) and validate(node.
8             right, node.val, max_val)
9     return validate(root, float('-inf'), float('inf'))

```

6.0.9 64. How do you find the diameter of a binary tree?

Compute max path through subtrees. Time complexity: $O(n)$.

```

1 def diameter_of_binary_tree(root):
2     max_diameter = [0]
3     def height(node):
4         if not node:
5             return 0
6         left = height(node.left)
7         right = height(node.right)
8         max_diameter[0] = max(max_diameter[0], left + right)
9         return max(left, right) + 1
10    height(root)
11    return max_diameter[0]

```

6.0.10 65. How do you implement a level-order traversal (BFS) of a binary tree?

Use a queue for level-by-level traversal. Time complexity: $O(n)$.

```

1 from collections import deque
2
3 def level_order(root):
4     if not root:
5         return []
6     result, queue = [], deque([root])
7     while queue:
8         level = []
9         for _ in range(len(queue)):
10            node = queue.popleft()
11            level.append(node.val)
12            if node.left:
13                queue.append(node.left)
14            if node.right:
15                queue.append(node.right)
16        result.append(level)
17    return result

```

7 Graphs

Graphs are non-linear structures with nodes and edges.

7.0.1 66. What is a graph?

A set of vertices connected by edges.

```

1 graph = {
2     'A': ['B', 'C'],
3     'B': ['A', 'D', 'E'],
4     'C': ['A', 'F'],
5     'D': ['B'],
6     'E': ['B', 'F'],
7     'F': ['C', 'E']
8 }

```

7.0.2 67. How do you implement a depth-first search (DFS) in a graph?

Use recursion or stack. Time complexity: $O(V + E)$.

```

1 def dfs(graph, start):
2     visited = set()
3     result = []
4     def dfs_helper(node):
5         visited.add(node)

```

```

6         result.append(node)
7         for neighbor in graph[node]:
8             if neighbor not in visited:
9                 dfs_helper(neighbor)
10    dfs_helper(start)
11    return result

```

7.0.3 68. How do you implement a breadth-first search (BFS) in a graph?

Use a queue for level-by-level traversal. Time complexity: $O(V + E)$.

```

1 from collections import deque
2
3 def bfs(graph, start):
4     visited = set()
5     queue = deque([start])
6     result = []
7     visited.add(start)
8     while queue:
9         node = queue.popleft()
10        result.append(node)
11        for neighbor in graph[node]:
12            if neighbor not in visited:
13                visited.add(neighbor)
14                queue.append(neighbor)
15    return result

```

7.0.4 69. How do you detect a cycle in a directed graph?

Use DFS with a recursion stack. Time complexity: $O(V + E)$.

```

1 def has_cycle(graph):
2     visited = set()
3     rec_stack = set()
4     def dfs(node):
5         visited.add(node)
6         rec_stack.add(node)
7         for neighbor in graph.get(node, []):
8             if neighbor not in visited:
9                 if dfs(neighbor):
10                    return True
11            elif neighbor in rec_stack:
12                return True
13        rec_stack.remove(node)
14        return False
15    for node in graph:
16        if node not in visited:
17            if dfs(node):
18                return True
19    return False

```

7.0.5 70. How do you find the shortest path in an unweighted graph?

Use BFS for layer-by-layer traversal. Time complexity: $O(V + E)$.

```

1 from collections import deque
2
3 def shortest_path(graph, start, end):
4     queue = deque([(start, [start])])
5     visited = {start}
6     while queue:

```

```

7     node, path = queue.popleft()
8     if node == end:
9         return path
10    for neighbor in graph[node]:
11        if neighbor not in visited:
12            visited.add(neighbor)
13            queue.append((neighbor, path + [neighbor]))
14    return []

```

7.0.6 71. How do you implement Dijkstra's algorithm for shortest paths?

Use a priority queue for minimum distance. Time complexity: $O((V + E) \log V)$.

```

1 import heapq
2
3 def dijkstra(graph, start):
4     distances = {node: float('inf') for node in graph}
5     distances[start] = 0
6     pq = [(0, start)]
7     while pq:
8         curr_dist, curr = heapq.heappop(pq)
9         if curr_dist > distances[curr]:
10             continue
11         for neighbor, weight in graph[curr].items():
12             distance = curr_dist + weight
13             if distance < distances[neighbor]:
14                 distances[neighbor] = distance
15                 heapq.heappush(pq, (distance, neighbor))
16    return distances

```

7.0.7 72. How do you perform a topological sort on a DAG?

Use DFS or Kahn's algorithm. Time complexity: $O(V + E)$.

```

1 def topological_sort(graph):
2     visited = set()
3     stack = []
4     def dfs(node):
5         visited.add(node)
6         for neighbor in graph.get(node, []):
7             if neighbor not in visited:
8                 dfs(neighbor)
9         stack.append(node)
10    for node in graph:
11        if node not in visited:
12            dfs(node)
13    return stack[::-1]

```

7.0.8 73. How do you find connected components in an undirected graph?

Use DFS or BFS to group nodes. Time complexity: $O(V + E)$.

```

1 def connected_components(graph):
2     visited = set()
3     components = []
4     def dfs(node, component):
5         visited.add(node)
6         component.append(node)
7         for neighbor in graph[node]:
8             if neighbor not in visited:
9                 dfs(neighbor, component)

```

```

10     for node in graph:
11         if node not in visited:
12             component = []
13             dfs(node, component)
14             components.append(component)
15     return components

```

7.0.9 74. How do you implement a minimum spanning tree (Kruskal's or Prim's)?

Use Prim's with a priority queue. Time complexity: $O((V + E) \log V)$.

```

1 import heapq
2
3 def prim_mst(graph):
4     visited = set()
5     mst = []
6     start = next(iter(graph))
7     pq = [(0, start, None)]
8     while pq:
9         weight, node, parent = heapq.heappop(pq)
10        if node in visited:
11            continue
12        visited.add(node)
13        if parent is not None:
14            mst.append((parent, node, weight))
15        for neighbor, w in graph[node].items():
16            if neighbor not in visited:
17                heapq.heappush(pq, (w, neighbor, node))
18    return mst

```

7.0.10 75. How do you check if a graph is bipartite?

Color nodes to ensure no adjacent nodes have the same color. Time complexity: $O(V + E)$.

```

1 from collections import deque
2
3 def is_bipartite(graph):
4     colors = {}
5     for node in graph:
6         if node not in colors:
7             colors[node] = 0
8             queue = deque([node])
9             while queue:
10                curr = queue.popleft()
11                for neighbor in graph[curr]:
12                    if neighbor not in colors:
13                        colors[neighbor] = 1 - colors[curr]
14                        queue.append(neighbor)
15                    elif colors[neighbor] == colors[curr]:
16                        return False
17    return True

```

8 Sorting

Sorting algorithms arrange elements in order.

8.0.1 76. What is bubble sort?

Swap adjacent elements if out of order. Time complexity: $O(n^2)$.

```

1 def bubble_sort(arr):
2     n = len(arr)
3     for i in range(n):
4         for j in range(0, n - i - 1):
5             if arr[j] > arr[j + 1]:
6                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
7     return arr

```

8.0.2 77. How does selection sort work?

Select smallest element and place at start. Time complexity: $O(n^2)$.

```

1 def selection_sort(arr):
2     for i in range(len(arr)):
3         min_idx = i
4         for j in range(i + 1, len(arr)):
5             if arr[j] < arr[min_idx]:
6                 min_idx = j
7         arr[i], arr[min_idx] = arr[min_idx], arr[i]
8     return arr

```

8.0.3 78. How does insertion sort work?

Insert elements into sorted portion. Time complexity: $O(n^2)$.

```

1 def insertion_sort(arr):
2     for i in range(1, len(arr)):
3         key = arr[i]
4         j = i - 1
5         while j >= 0 and arr[j] > key:
6             arr[j + 1] = arr[j]
7             j -= 1
8         arr[j + 1] = key
9     return arr

```

8.0.4 79. How does merge sort work?

Divide, sort, and merge halves. Time complexity: $O(n \log n)$.

```

1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     left = merge_sort(arr[:mid])
6     right = merge_sort(arr[mid:])
7     return merge(left, right)
8
9 def merge(left, right):
10    result = []
11    i = j = 0
12    while i < len(left) and j < len(right):
13        if left[i] <= right[j]:
14            result.append(left[i])
15            i += 1
16        else:
17            result.append(right[j])
18            j += 1
19    result.extend(left[i:])
20    result.extend(right[j:])
21    return result

```

8.0.5 80. How does quicksort work?

Partition around a pivot and sort subarrays. Time complexity: $O(n \log n)$ average.

```
1 def quicksort(arr, low, high):
2     if low < high:
3         pi = partition(arr, low, high)
4         quicksort(arr, low, pi - 1)
5         quicksort(arr, pi + 1, high)
6
7 def partition(arr, low, high):
8     pivot = arr[high]
9     i = low - 1
10    for j in range(low, high):
11        if arr[j] <= pivot:
12            i += 1
13            arr[i], arr[j] = arr[j], arr[i]
14    arr[i + 1], arr[high] = arr[high], arr[i + 1]
15    return i + 1
```

8.0.6 81. What is heap sort?

Build max-heap and extract maximum. Time complexity: $O(n \log n)$.

```
1 import heapq
2
3 def heap_sort(arr):
4     heapq.heapify(arr)
5     result = []
6     while arr:
7         result.append(heapq.heappop(arr))
8     return result
```

8.0.7 82. How does counting sort work?

Use a count array for frequencies. Time complexity: $O(n + k)$.

```
1 def counting_sort(arr):
2     max_val = max(arr)
3     count = [0] * (max_val + 1)
4     for num in arr:
5         count[num] += 1
6     result = []
7     for i in range(len(count)):
8         result.extend([i] * count[i])
9     return result
```

8.0.8 83. How does radix sort work?

Sort digits using counting sort. Time complexity: $O(d(n + k))$.

```
1 def radix_sort(arr):
2     max_val = max(arr)
3     exp = 1
4     while max_val // exp > 0:
5         counting_sort_by_digit(arr, exp)
6         exp *= 10
7
8 def counting_sort_by_digit(arr, exp):
9     n = len(arr)
10    output = [0] * n
11    count = [0] * 10
```

```

12     for i in range(n):
13         index = arr[i] // exp
14         count[index % 10] += 1
15     for i in range(1, 10):
16         count[i] += count[i - 1]
17     for i in range(n - 1, -1, -1):
18         index = arr[i] // exp
19         output[count[index % 10] - 1] = arr[i]
20         count[index % 10] -= 1
21     for i in range(n):
22         arr[i] = output[i]

```

8.0.9 84. What is the difference between stable and unstable sorting?

Stable sorting preserves relative order of equal elements.

```

1 # Example: Stable sort (merge sort preserves order)
2 arr = [(1, 'a'), (1, 'b')]
3 arr = merge_sort(arr) # Preserves order of equal keys

```

8.0.10 85. How do you sort an array with elements in a given range?

Use counting sort for efficiency. Time complexity: $O(n + k)$.

```

1 def sort_in_range(arr, k):
2     count = [0] * (k + 1)
3     for num in arr:
4         count[num] += 1
5     result = []
6     for i in range(k + 1):
7         result.extend([i] * count[i])
8     return result

```

9 Searching

Searching algorithms locate elements efficiently.

9.0.1 86. How does linear search work?

Iterate until target is found. Time complexity: $O(n)$.

```

1 def linear_search(arr, target):
2     for i in range(len(arr)):
3         if arr[i] == target:
4             return i
5     return -1

```

9.0.2 87. How does binary search work?

Divide sorted array in half. Time complexity: $O(\log n)$.

```

1 def binary_search(arr, target):
2     left, right = 0, len(arr) - 1
3     while left <= right:
4         mid = (left + right) // 2
5         if arr[mid] == target:
6             return mid
7         elif arr[mid] < target:
8             left = mid + 1
9         else:

```

```
10         right = mid - 1
11     return -1
```

9.0.3 88. How do you implement a binary search iteratively?

Use a loop for binary search. Time complexity: $O(\log n)$.

```
1 def binary_search_iterative(arr, target):
2     left, right = 0, len(arr) - 1
3     while left <= right:
4         mid = (left + right) // 2
5         if arr[mid] == target:
6             return mid
7         elif arr[mid] < target:
8             left = mid + 1
9         else:
10            right = mid - 1
11    return -1
```

9.0.4 89. How do you search in a rotated sorted array?

Modify binary search for rotation. Time complexity: $O(\log n)$.

```
1 def search_rotated_array(arr, target):
2     left, right = 0, len(arr) - 1
3     while left <= right:
4         mid = (left + right) // 2
5         if arr[mid] == target:
6             return mid
7         if arr[left] <= arr[mid]:
8             if arr[left] <= target < arr[mid]:
9                 right = mid - 1
10            else:
11                left = mid + 1
12        else:
13            if arr[mid] < target <= arr[right]:
14                left = mid + 1
15            else:
16                right = mid - 1
17    return -1
```

9.0.5 90. How do you find the first and last occurrence of an element in a sorted array?

Use binary search for boundaries. Time complexity: $O(\log n)$.

```
1 def find_first_last(arr, target):
2     def find_first(arr, target):
3         left, right = 0, len(arr) - 1
4         first = -1
5         while left <= right:
6             mid = (left + right) // 2
7             if arr[mid] == target:
8                 first = mid
9                 right = mid - 1
10            elif arr[mid] < target:
11                left = mid + 1
12            else:
13                right = mid - 1
14        return first
15
16    def find_last(arr, target):
```

```

17     left, right = 0, len(arr) - 1
18     last = -1
19     while left <= right:
20         mid = (left + right) // 2
21         if arr[mid] == target:
22             last = mid
23             left = mid + 1
24         elif arr[mid] < target:
25             left = mid + 1
26         else:
27             right = mid - 1
28     return last
29
30     return [find_first(arr, target), find_last(arr, target)]

```

10 Dynamic Programming

Dynamic programming solves problems by storing subproblem solutions.

10.0.1 91. What is dynamic programming?

A method to avoid recomputation by storing subproblem solutions.

```

1 # Example: Memoization
2 def fib(n, memo={}):
3     if n in memo:
4         return memo[n]
5     if n <= 1:
6         return n
7     memo[n] = fib(n - 1, memo) + fib(n - 2, memo)
8     return memo[n]

```

10.0.2 92. How do you solve the Fibonacci sequence using dynamic programming?

Use a table iteratively. Time complexity: $O(n)$.

```

1 def fibonacci(n):
2     if n <= 1:
3         return n
4     dp = [0] * (n + 1)
5     dp[1] = 1
6     for i in range(2, n + 1):
7         dp[i] = dp[i - 1] + dp[i - 2]
8     return dp[n]

```

10.0.3 93. How do you solve the 0/1 knapsack problem?

Use a 2D table for max value. Time complexity: $O(nW)$.

```

1 def knapsack(values, weights, W):
2     n = len(values)
3     dp = [[0] * (W + 1) for _ in range(n + 1)]
4     for i in range(1, n + 1):
5         for w in range(W + 1):
6             if weights[i - 1] <= w:
7                 dp[i][w] = max(dp[i - 1][w], dp[i - 1][w - weights[i - 1]]
                                + values[i - 1])
8             else:
9                 dp[i][w] = dp[i - 1][w]
10    return dp[n][W]

```

10.0.4 94. How do you find the longest common subsequence of two strings?

Use a 2D table to compare characters. Time complexity: $O(mn)$.

```
1 def longest_common_subsequence(s1, s2):
2     m, n = len(s1), len(s2)
3     dp = [[0] * (n + 1) for _ in range(m + 1)]
4     for i in range(1, m + 1):
5         for j in range(1, n + 1):
6             if s1[i - 1] == s2[j - 1]:
7                 dp[i][j] = dp[i - 1][j - 1] + 1
8             else:
9                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
10    return dp[m][n]
```

10.0.5 95. How do you solve the coin change problem?

Track minimum coins for each amount. Time complexity: $O(n * \text{amount})$.

```
1 def coin_change(coins, amount):
2     dp = [float('inf')] * (amount + 1)
3     dp[0] = 0
4     for i in range(1, amount + 1):
5         for coin in coins:
6             if coin <= i:
7                 dp[i] = min(dp[i], dp[i - coin] + 1)
8     return dp[amount] if dp[amount] != float('inf') else -1
```

10.0.6 96. How do you solve the longest increasing subsequence problem?

Track lengths of increasing subsequences. Time complexity: $O(n^2)$.

```
1 def longest_increasing_subsequence(arr):
2     dp = [1] * len(arr)
3     for i in range(1, len(arr)):
4         for j in range(i):
5             if arr[i] > arr[j]:
6                 dp[i] = max(dp[i], dp[j] + 1)
7     return max(dp)
```

10.0.7 97. How do you solve the matrix chain multiplication problem?

Find minimum cost of multiplying matrices. Time complexity: $O(n^3)$.

```
1 def matrix_chain_multiplication(dimensions):
2     n = len(dimensions) - 1
3     dp = [[0] * n for _ in range(n)]
4     for length in range(2, n + 1):
5         for i in range(n - length + 1):
6             j = i + length - 1
7             dp[i][j] = float('inf')
8             for k in range(i, j):
9                 cost = dp[i][k] + dp[k + 1][j] + dimensions[i] * dimensions
10                    [k + 1] * dimensions[j + 1]
11                dp[i][j] = min(dp[i][j], cost)
12    return dp[0][n - 1]
```

10.0.8 98. How do you solve the edit distance problem?

Compute minimum operations to transform strings. Time complexity: $O(mn)$.

```

1 def edit_distance(s1, s2):
2     m, n = len(s1), len(s2)
3     dp = [[0] * (n + 1) for _ in range(m + 1)]
4     for i in range(m + 1):
5         dp[i][0] = i
6     for j in range(n + 1):
7         dp[0][j] = j
8     for i in range(1, m + 1):
9         for j in range(1, n + 1):
10            if s1[i - 1] == s2[j - 1]:
11                dp[i][j] = dp[i - 1][j - 1]
12            else:
13                dp[i][j] = min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]) + 1
14     return dp[m][n]

```

10.0.9 99. How do you solve the subset sum problem?

Check if a subset sums to target. Time complexity: $O(n * \text{sum})$.

```

1 def subset_sum(arr, target):
2     n = len(arr)
3     dp = [[False] * (target + 1) for _ in range(n + 1)]
4     for i in range(n + 1):
5         dp[i][0] = True
6     for i in range(1, n + 1):
7         for j in range(1, target + 1):
8             if arr[i - 1] <= j:
9                 dp[i][j] = dp[i - 1][j - arr[i - 1]] or dp[i - 1][j]
10            else:
11                dp[i][j] = dp[i - 1][j]
12     return dp[n][target]

```

10.0.10 100. How do you solve the rod cutting problem?

Find maximum revenue from cutting a rod. Time complexity: $O(n^2)$.

```

1 def rod_cutting(prices, n):
2     dp = [0] * (n + 1)
3     for i in range(1, n + 1):
4         max_val = float('-inf')
5         for j in range(i):
6             max_val = max(max_val, prices[j] + dp[i - j - 1])
7         dp[i] = max_val
8     return dp[n]

```